Lab No: 14

❋ Lab Title: Design and implementation of relative grading system

Learning objectives:

The objective of this open-ended lab is to design and implement a complete **Relative Grading Based Student Transcript System** using C++ programming language. The system is developed to simulate a real university academic management environment where student records, grades, CGPA, and ranking are managed automatically.

The program aims to:

- Store and manage records of 50 students
- Implement a relative grading system
- Calculate CGPA using credit hours
- Generate student transcripts dynamically
- Assign class ranking based on academic performance
- Demonstrate practical use of structures, arrays, functions, loops, and formatted output in C++

Introduction:

Modern universities use automated transcript management systems to efficiently handle student academic data. These systems maintain student records, calculate GPA/CGPA, assign grades, and generate transcripts.

In this project, a console-based Student Transcript System has been developed using C++. The system follows a **relative grading approach**, where grades are assigned according to performance ranges rather than fixed percentages only.

The project demonstrates how programming concepts can be applied to solve real-world academic management problems.

➢ Theory:

1: Structure in C++

A structure (`struct`) is a user-defined data type that groups multiple related variables under a single name.

In this project, structures are used to represent student information such as:

- Registration number
- Student name
- Subject marks
- Grades
- CGPA
- Rank

Example:

```
struct Student
{
    int regNo;
    string name;
    int marks[8];
    string grades[8];
    float cgpa;
    int rank;
};
```

➢ Relative grading system:

Relative grading is a grading technique in which student performance is evaluated comparatively rather than absolutely.

The grading scale used in this project is:

This grading system reflects relative academic performance of students.

Marks Range	Grade	GPA
85 - 100	A+	4.0
75 - 84	A	3.7
65 - 74	B+	3.3
55 - 64	B	3.0
45 - 54	C+	2.7
40 - 44	C	2.3
35 - 39	D	2.0
Below 35	F	0.0

➢ CGPA Calculation:

CGPA is calculated using subject credit hours.

Formula:

CGPA=∑(Grade Points×Credit Hours)/∑Credit Hours

The system multiplies GPA points with corresponding credit hours and divides by total credit hours.

Array of structure:

The program stores records of 50 students using an array of structures:

Student s[50];

This allows efficient storage and retrieval of multiple student records.

➢ Linear Search Algorithm:

The transcript system uses linear search to locate a student record using registration number.

Algorithm:

1. Input registration number
2. Traverse all student records
3. Compare entered registration number with stored records
4. Display transcript if match found
5. Otherwise display “Student Not Found”

➢ Tools and environment:

Tool / Software	Description
Programming Language	C++
Compiler	GCC / G++
IDE	Code::Blocks / Dev-C++ / VS Code
Libraries Used	<iostream>, <string>, <iomanip>
Operating System	Windows 10 / 11

Program Design

The program consists of the following major components:

1. Student Structure

Stores all student-related information.

2. Grade Conversion Function

Converts marks into relative grades.

3. GPA Conversion Function

Converts grades into GPA points.

4. CGPA Calculation Module

Calculates weighted CGPA using credit hours.

5. Ranking System

Determines class rank according to CGPA.

6. Transcript Display Module

Displays formatted student transcript.

- **Header files:**

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
```

- **Student structure:**

```
struct Student
{
    int regNo;
    string name;
    int marks[8];
    string grades[8];
    float cgpa;
    int rank;
};
```

- **Relative grade function:**

```
string getGrade(int m)
{
    if(m >= 85) return "A+";
    else if(m >= 75) return "A";
    else if(m >= 65) return "B+";
    else if(m >= 55) return "B";
    else if(m >= 45) return "C+";
    else if(m >= 40) return "C";
    else if(m >= 35) return "D";
    else return "F";
}
```

CGPA Calculation function

```
float getPoint(string g)
{
    if(g=="A+") return 4.0;
    if(g=="A") return 3.7;
    if(g=="B+") return 3.3;
    if(g=="B") return 3.0;
    if(g=="C+") return 2.7;
    if(g=="C") return 2.3;
    if(g=="D") return 2.0;
    return 0.0;
}
```

- **Algorithm:**

  - START

- Define student structure
- Initialize 50 student records
- Generate subject marks automatically
- Assign relative grades
- Convert grades into GPA points
- Calculate CGPA using credit hours
- Compute student ranking
- Input registration number
- Search student record
- Display transcript
- END

↳ Complete program :

```
#include <iostream>     // Input and Output library
#include <string>       // String library
#include <iomanip>      // Formatting library (setw, setprecision)

using namespace std;    // Standard namespace

// Constant variable for total students
const int SIZE = 50;

// Student structure
struct Student
{
   int regNo;          // Registration number
   string name;        // Student name

   int marks[8];       // Array for subject marks
   string grades[8];   // Array for subject grades

   float cgpa;         // Student CGPA
   int rank;           // Student class rank
};

// Subject names array
string subjects[8] =
{
   "English",
   "PakStudy",
   "Mechanics",
   "Linear Algebra",
   "Programming",
```

```
    "Programming Lab",
    "Workshop",
    "Fahmi Quran"
};

// Credit hours array
int credit[8] = {3,2,3,3,3,1,1,2};

// Function to convert marks into grades
string getGrade(int m)
{
    // If marks greater than or equal to 85
    if(m >= 85)
        return "A+";

    // If marks between 75 and 84
    else if(m >= 75)
        return "A";

    // If marks between 65 and 74
    else if(m >= 65)
        return "B+";

    // If marks between 55 and 64
    else if(m >= 55)
        return "B";

    // If marks between 45 and 54
    else if(m >= 45)
        return "C+";

    // If marks between 40 and 44
    else if(m >= 40)
        return "C";

    // If marks between 35 and 39
    else if(m >= 35)
        return "D";

    // If marks below 35
    else
        return "F";
}

// Function to convert grades into GPA points
float getPoint(string g)
{
    // GPA for A+
    if(g == "A+")
        return 4.0;

    // GPA for A
    else if(g == "A")
        return 3.7;

    // GPA for B+
```

```
    else if(g == "B+")
        return 3.3;

    // GPA for B
    else if(g == "B")
        return 3.0;

    // GPA for C+
    else if(g == "C+")
        return 2.7;

    // GPA for C
    else if(g == "C")
        return 2.3;

    // GPA for D
    else if(g == "D")
        return 2.0;

    // GPA for F
    else
        return 0.0;
}

// Main function starts here
int main()
{
    // Create array of 50 students
    Student s[SIZE];

    // Array containing 50 student names
    string names[50] =
    {
        "Ali","Ahmed","Usman","Hamza","Bilal",
        "Zain","Hassan","Fahad","Saad","Omar",
        "Yasir","Asad","Shahzaib","Talha","Abdullah",
        "Ayan","Rizwan","Irfan","Danish","Kamran",
        "Noman","Sufyan","Taha","Huzaifa","Ahsan",
        "Imran","Zeeshan","Arslan","Rehan","Junaid",
        "Kashif","Waqas","Shahbaz","Farhan","Adil",
        "Mudassar","Faisal","Naveed","Saif","Haris",
        "Fawad","Musa","Raza","Khalid","Nadir",
        "Sami","Yousaf","Adeel","Ibrahim","Shayan"
    };

    // Loop to generate data for all students
    for(int i = 0; i < SIZE; i++)
    {
        // Generate registration number
        s[i].regNo = 3001 + i;

        // Assign student name
        s[i].name = names[i];

        // Loop for marks and grades
        for(int j = 0; j < 8; j++)
```

```
        {
            // Generate marks automatically
            s[i].marks[j] = 40 + ((i * 7 + j * 5) % 60);

            // Convert marks into grades
            s[i].grades[j] = getGrade(s[i].marks[j]);
        }

        // Variables for CGPA calculation
        float totalPoints = 0;
        float totalCredits = 0;

        // Loop for GPA calculation
        for(int j = 0; j < 8; j++)
        {
            // Multiply GPA with credit hours
            totalPoints += getPoint(s[i].grades[j]) * credit[j];

            // Add total credit hours
            totalCredits += credit[j];
        }

        // Final CGPA formula
        s[i].cgpa = totalPoints / totalCredits;
    }

    // Rank calculation loop
    for(int i = 0; i < SIZE; i++)
    {
        // Initial rank
        int r = 1;

        // Compare CGPA with all students
        for(int j = 0; j < SIZE; j++)
        {
            // If another student has higher CGPA
            if(s[j].cgpa > s[i].cgpa)
            {
                // Increase rank
                r++;
            }
        }

        // Assign final rank
        s[i].rank = r;
    }

    // Variable for registration number input
    int regNo;

    // Boolean variable to check student found or not
    bool found = false;

    // University Header
    cout                                                    <<
"=====================================================\n";
```

```
    cout << "   UNIVERSITY OF ENGINEERING AND TECHNOLOGY\n";

    cout << "   Department of Electrical AI\n";

    cout                                                                    <<
"=========================================================\n";

    // Input registration number
    cout << "\nEnter Registration Number: ";

    cin >> regNo;

    // Search student loop
    for(int i = 0; i < SIZE; i++)
    {
        // If registration number matched
        if(regNo == s[i].regNo)
        {
            // Student found
            found = true;

            // Transcript heading
            cout            <<          "\n================          TRANSCRIPT
================\n";

            // Print student name
            cout << "Name     : " << s[i].name << endl;

            // Print registration number
            cout << "Reg No   : " << s[i].regNo << endl;

            cout << "\n";

            // Table heading
            cout << left
                 << setw(22) << "Subject"
                 << setw(10) << "Marks"
                 << setw(10) << "Grade"
                 << setw(10) << "CH" << endl;

            // Table line
            cout << "-------------------------------------------------\n";

            // Loop to display all subjects
            for(int j = 0; j < 8; j++)
            {
                cout << left
                     << setw(22) << subjects[j]
                     << setw(10) << s[i].marks[j]
                     << setw(10) << s[i].grades[j]
                     << setw(10) << credit[j]
                     << endl;
            }

            // Bottom line
```

```
            cout << "-----------------------------------------------\n";

            // Set decimal precision to 2
            cout << fixed << setprecision(2);

            // Print CGPA
            cout << "\nCGPA : " << s[i].cgpa << endl;

            // Print Rank
            cout << "Rank : " << s[i].rank << endl;

            // Ending line
            cout                                                    <<
"\n=================================================\n";
        }
    }

    // If student not found
    if(found == false)
    {
        cout << "\nStudent Record Not Found!\n";
    }

    // Program ends successfully
    return 0;
}
```

➧ Output:

```
UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT: ELECTRICAL AI
=========================================

Enter Registration Number: 3007

============= TRANSCRIPT =============
Reg No : 3007
Name   : Yasir Shah

Subject            Marks   Grade
English            78      A
Pak Study          62      B+
Mechanics          70      B+
Linear Algebra     68      B+
Programming        80      A
Programming Lab    85      A+
Workshop           55      B
Fahmi Quran        65      B+

CGPA : 3.65
Rank : 4
=========================================
```

Discussion:

This project demonstrates practical implementation of important C++ programming concepts in a real-world academic scenario. The use of structures and arrays simplifies data organization, while functions improve modularity and maintainability.

The relative grading approach provides a more realistic university-level grading system. The project can be further enhanced by adding:

- File handling
- Database connectivity
- GUI interface
- Dynamic student data entry
- Attendance and fee management modules

conclusion:

This open-ended lab successfully implemented a complete **Relative Grading Based Student Transcript System** using C++. The system efficiently manages student academic records, calculates CGPA using credit hours, assigns relative grades, and determines class ranking.

The project provided strong practical understanding of:

- Structures
- Arrays
- Functions
- Searching algorithms
- Relative grading systems
- Data management techniques

This system represents a simplified but realistic university transcript management application.
